

Auditoría técnica del backend (FastAPI + SQLAlchemy + PostgreSQL)

Revisé todo el código fuente, las 5 migraciones de Alembic, los tests, el Dockerfile y los scripts ETL. Dejé fuera __pycache__ y .pytest_cache. No pude ejecutar la suite porque mi entorno no tiene las dependencias ni red. Todo es análisis estático, y marco como "a verificar" lo que conviene confirmar con un test. No incluyo la contraseña del .env que viene dentro del zip.

1. Resumen ejecutivo

Estado general. El backend tiene buena base: arquitectura en capas legible, validación de entrada seria, defensas contra IDOR y fugas de PII, y reglas de dominio razonables. Sin embargo, **hoy no funcionaría en producción**. Hay tres bloqueos independientes:

1. **Las migraciones no coinciden con los modelos.** Una base creada con alembic upgrade head rompe los mazos, el registro sin teléfono y todo el módulo de trade.
2. **El login por OTP no puede completarse.** El código nunca se envía y, además, hay un error de comparación de fechas que da 500.
3. **Las propuestas de intercambio no validan propiedad de las cartas y complete no mueve inventario.**

Fortalezas:

- Configuración fail-fast con ENVIRONMENT en production por defecto.
- OTP con HMAC y comparación en tiempo constante.
- Filtros por user_id en servicios (anti-IDOR con 404 opaco).
- Separación de schemas público y privado.
- ILIKE siempre escapado.
- Agregados con invariantes (Collection, Deck).
- Uso razonable de Strategy.
- Upserts en las ingestas.

Riesgo general: alto, concentrado en integridad de datos y autenticación. La arquitectura en sí es sana.

Qué falta para ser profesional:

- Migraciones verificadas contra los modelos.
 - Envío real de OTP.
 - Máquina de estados del trade.
 - Tests contra PostgreSQL.
 - README, CI y dependencias fijadas.
 - Configuración de logs, pool y rate limit para varias instancias.
-

2. Arquitectura

Es una **arquitectura en capas con service layer** y un toque DDD ligero. Los modelos ORM son a la vez entidades de dominio con comportamiento (patrón Active Record). No es Clean ni Hexagonal, porque el dominio depende de SQLAlchemy y los servicios lanzan HTTPException.

Cliente

- └ FastAPI (main.py: CORS, slowapi, handler IntegrityError, /health/*)
- └ routers/* HTTP + Depends(get_db, get_current_user)
- └ services/* classmethods: reglas + db.query + db.commit + HTTPException
 - └ models/* ORM + comportamiento (Collection, Deck, User, UserCard)
 - └ repositories/card_repository.py (solo catálogo)
- └ schemas/* DTOs Pydantic

core/{config,security,limiter} scripts/ (ETL) → modelos directo

- └ PostgreSQL (SQLAlchemy síncrono, psycopg2)

Preguntas clave:

1. **¿Responsabilidades bien distribuidas?** En general sí. Las excepciones están en la tabla de la sección 3.
2. **¿Lógica de negocio en controladores?** No, los routers son delgados. ●

3. **¿Acceso a BD donde no debe?** Los servicios usan db.query directo y solo el catálogo tiene repositorio. Es un patrón a medias, pero a este tamaño no recomiendo crear repositorios para todo (YAGNI).
 4. **¿Se podría cambiar una parte sin romper otras?** En su mayoría sí. Dos acoplamientos lo impiden:
 - Los servicios acoplados a HTTP (HTTPException dentro de services/), lo que impide reutilizarlos desde scripts o jobs.
 - Los lazy="joined" globales en los modelos, que hacen que cualquier consulta cargue JSONB.
 5. **Transacciones:** los servicios llaman a db.commit() internamente. No hay frontera de Unit of Work, así que las operaciones no se pueden componer.
-

3. POO y SOLID

Principio	Estado	Ubicación	Problema	Solución
S		repositories/card_repository.py	El repositorio mezcla persistencia con heurística de dominio (MECHANIC_RULES, score_candidate)	Mover la heurística a services/card_similarity.py; el repositorio solo trae candidatos
S		models/deck.py (319 líneas)	Un módulo con adapter, strategies, factory y 2 modelos ORM	Separar en domain/deck_rules.py y models/deck.py
S		models/user.py	User concentra identidad, OTP, reputación y	Usar el método del modelo desde

Principio	Estado	Ubicación	Problema	Solución
			perfil. update_profile y mark_phone_as_ verified existen pero nadie los usa (el servicio hace setattr en bucle)	UserService o eliminarlo
O	 / 	LegalityStrategyFactory, scryfall_query_parser.py	La factory es un registro correcto. El parser es una cadena if/elif por clave	Registro dict[str, handler] si crecen los operadores
L		deck.py: DeckLegalityStrategy	Base con raise NotImplemented Error en vez de ABC. max_allowed_in_ deck declara Optional[int] pero devuelve float("inf")	ABC + @abstractmethod; tipar como `float
I		N/A	No hay interfaces que segregar	Cuando se cree OtpSender, mantenerla de un solo método
D		services/*	Dependen de Session, ORM y HTTPException. settings es un singleton global	Excepciones de dominio + un handler que las traduzca a HTTP. No hace falta abstraer Session

Principio	Estado	Ubicación	Problema	Solución
Encapsulación		trade_proposal_service.py:145,159,174	proposal.status = ... se asigna desde el servicio, sin pasar por el agregado. Collection y Deck son ricos pero TradeProposal es anémico	Métodos accept(), reject(), complete() con tabla de transiciones
Adapter		deck.py:67-73	MTGCardDomainAdapter usa getattr(..., "color_identity", []) y oculta que ese atributo no existe en CartaScryfall (ver 4.4)	Leer de scryfall_raw_data["color_identity"] o añadir la columna

Patrones:

- **Bien usados:** Service layer, Strategy (legalidad), DTO/Pydantic, Facade (find_trade_matches_for_user), upsert ON CONFLICT.
- **A medias:** Repository.
- **Mal nombrado:** MatchedCardSanitizer en realidad es un mapper.
- **Estado global:** DynamicVocabularyCache usa estado de clase global, y es por proceso.
- **Patrones que sí añadiría (solo estos):**
 - Una interfaz OtpSender (consola en dev, proveedor en prod).
 - Una tabla de transiciones para TradeStatus.

- Unit of Work a nivel de dependencia (commit/rollback en get_db).

4. Problemas críticos ●

4.1 Migraciones desfasadas de los modelos

Los tests usan create_all sobre SQLite, así que nunca lo ven. Esto aplica si la BD se crea con Alembic, como implican el Dockerfile y el alembic.ini. Ningún archivo del proyecto lo documenta.

Desfase	Efecto
No existe migración de trade_proposals ni trade_proposal_items. Además models/__init__.py no importa TradeProposal, así que --autogenerate tampoco los ve	Todo /api/trade/* da 500
Deck.featured_card_id (deck.py:209) no está en ninguna migración	Toda consulta a Deck falla (el SELECT incluye la columna)
users.phone_number es NOT NULL en 747f7bc41787, pero modelo y schema lo declaran opcional	El registro sin teléfono lanza IntegrityError, que el handler global convierte en un 409 "cuenta existente" engañoso
cartas.scryfall_raw_data es JSON en la migración y JSONB en el modelo	jsonb_array_elements_text(json) no existe. El except Exception: return [] de card_catalog_service.py:67 lo oculta y el autocompletado de keywords queda vacío
uq_collection_physical_card_instance (UserCard) no está en migraciones	Se pueden duplicar copias físicas
user_cards.created_at está en la migración pero no en el modelo	hasattr(UserCard, "created_at") es False, así que se ordena por UUID aleatorio
NOT NULL en el modelo y nullable en BD (users.*, decks.format,	Deriva silenciosa

Desfase

Efecto

deck_cards.category). Faltan
ix_collections_is_public_trade y
ix_cartas_colors

0365dbc64f46 añade otp_attempts NOT
NULL sin default

Falla con filas existentes (parchado
después en c4a11b9d0e23)

Solución:

1. Importar todos los modelos en models/__init__.py.
2. Generar una migración correctiva con alembic revision --autogenerate y revisarla a mano.
3. Añadir un test en CI: alembic upgrade head sobre PostgreSQL seguido de alembic check.

4.2 El login por OTP no es utilizable

a) El código nunca se envía. AuthService.request_otp genera y guarda el OTP, pero no hay ningún código de envío de email o SMS. Solo existe dev_otp_code, y Settings prohíbe EXPOSE_DEV_OTP en producción. En producción nadie puede autenticarse.

b) Comparación de fechas naive vs aware (models/user.py:56-79; a verificar con un test, aunque es determinista). can_request_otp e is_otp_challenge_valid comparan datetime.now(timezone.utc) con otp_expires_at. La columna es DateTime sin zona horaria, así que psycpg2 la devuelve naive. El resultado es TypeError → 500 al verificar y al re-solicitar. security.py:46-50 sí normaliza, pero el modelo no. Los tests no lo detectan porque usan MagicMock y no hay ningún flujo request → verify.

c) Contadores de intentos inconsistentes. verify_otp_digest suma 1 al intento (security.py:59) y luego register_failed_otp_attempt suma otro, así que cada fallo cuenta 2. Los límites además difieren: 5 en security.py y 3 en user.py. El resultado es que un usuario legítimo queda bloqueado tras 2 errores, y la rama de 5 intentos es inalcanzable. Con solo conocer el correo de la víctima, un atacante puede bloquearlo el login repetidamente, porque el rate limit es solo por IP.

Corrección (un único punto de verdad en el modelo):

```
python
```

```
# models/user.py
```

OTP_MAX_ATTEMPTS = 5

```
def _utc(dt):
```

```
    return dt if dt is None or dt.tzinfo else dt.replace(tzinfo=timezone.utc)
```

```
def verify_otp(self, code: str) -> bool:
```

```
    expires = _utc(self.otp_expires_at)
```

```
    if (not self.otp_hash or not expires
```

```
        or expires < datetime.now(timezone.utc)
```

```
        or self.otp_attempts >= OTP_MAX_ATTEMPTS):
```

```
        return False
```

```
    self.otp_attempts += 1
```

```
    ok = hmac.compare_digest(self.otp_hash, hash_otp(code, str(self.id)))
```

```
    if ok:
```

```
        self.clear_otp_challenge()
```

```
    return ok
```

AuthService.verify_otp_and_login solo llama a user.verify_otp(code) y hace commit en ambos casos. A medio plazo, cambiar las columnas a DateTime(timezone=True). Para el envío, una interfaz OtpSender inyectada.

4.3 Propuestas de trade sin validación de propiedad ni máquina de estados

trade_proposal_service.py:

- **create_proposal (51-109)** solo comprueba UserCard.id y quantity >=. Un usuario puede:
 - ofrecer cartas que no son suyas;
 - pedir cartas que no pertenecen al receptor, que están en binders privados o que no están marcadas is_for_trade.

Tampoco valida:

- que el receptor exista;
 - que cash_payer_id sea proponente o receptor;
 - la moneda;
 - listas vacías o duplicadas;
 - la longitud de notes.
- **complete_trade (165-182)** dice en el docstring "ejecuta el impacto en inventarios", pero no mueve ninguna carta y solo suma reputación. Cualquiera de los dos participantes puede completarlo unilateralmente, así que se puede farmear reputación con dos cuentas propias (+2 por trade, tope 200).
 - **reject_proposal** permite que cualquiera de los dos rechace incluso una propuesta ya ACCEPTED. COUNTERED y CANCELLED existen en el enum pero no tienen endpoint.
 - **Carreras:** no hay with_for_update. Un doble complete suma la reputación dos veces (register_successful_trade es read-modify-write), y la misma carta puede estar comprometida en N propuestas.

Corrección mínima:

python

```
_TRANSITIONS = {
    TradeStatus.PROPOSED: {TradeStatus.ACCEPTED, TradeStatus.REJECTED,
TradeStatus.CANCELLED},
    TradeStatus.ACCEPTED: {TradeStatus.COMPLETED, TradeStatus.CANCELLED},
}
```

```
def _load_cards(db, items, owner_id, *, public_only):
    ids = [i.user_card_id for i in items]
    if len(set(ids)) != len(ids):
        raise DomainError("Cartas duplicadas.")
    q = (db.query(UserCard).join(Collection)
        .filter(UserCard.id.in_(ids), Collection.user_id == owner_id))
```

```

if public_only:

    q = q.filter(UserCard.is_for_trade.is_(True), Collection.is_public_trade.is_(True))

    by_id = {r.id: r for r in q.with_for_update().all()}

    for it in items:

        uc = by_id.get(it.user_card_id)

        if not uc or uc.quantity < it.quantity:

            raise DomainError("Carta no disponible.")

    return by_id

```

offered → owner=proposer, public_only=False; requested → owner=receiver,
public_only=True

Además: que complete requiera confirmación de ambas partes (o al menos transfiera inventario en la misma transacción) y que el servicio ejecute transiciones solo vía TradeProposal.accept()/reject()/complete().

4.4 La validación de identidad de color de Commander no hace nada ●

Lo clasifico como importante, pero va aquí por ser un error silencioso. CartaScryfall no tiene color_identity (solo colors, ver models/card.py), así que el adapter devuelve set() siempre. card.color_identity.issubset(cmd_identity) es True para cualquier carta y la regla nunca se dispara.

Los tests pasan porque create_mock_catalog_card inventa card.color_identity en un MagicMock, lo que da falsa confianza.

Solución: guardar color_identity como columna en la ingesta (o leerlo de scryfall_raw_data), y probar con instancias reales de CartaScryfall en vez de mocks.

5. Problemas importantes ● ●

● **Eager loading que arrastra JSONB.** Deck.cards es lazy="joined" (deck.py:212-217), y DeckCard.card_catalog, UserCard.card_catalog (collection.py:55) y TradeProposalItem.user_card también. Collection.cards es selectin (collection.py:100-105). Cada carga de entidad trae scryfall_raw_data (varios KB por carta) aunque el schema no lo use. Casos concretos:

- /decks/me carga todas las cartas de todos los mazos.

- /collections/me carga todas las cartas de todos los binders solo para devolver metadatos.
- UserService.get_profile_aggregates (user_service.py:64-84) hace una consulta agregada "para evitar N+1", pero al cargar la entidad Collection dispara el selectin de todas sus cartas.
- El matchmaking (_find_counterparts_with_my_wants) no hace defer del JSONB y no limita antes de recortar con [:limit].

python

card.py

scryfall_raw_data = deferred(Column(JSONB, nullable=False))

collection.py / deck.py: lazy="select" (o "raise") y options() explícitos donde se necesiten

● Endpoints públicos costosos y sin rate limit.

- /cards/search con o:, a: o kw: hace ILIKE sobre oracle_text y sobre JSONB, sin índice (solo name tiene trigram).
- /cards/autocomplete?field=artist|oracle tiene el mismo problema.
- /cards/{id}/similar carga 100 filas con JSONB completo, aplica regex en Python y no usa caché.
- DynamicVocabularyProvider hace un SELECT DISTINCT sobre toda la tabla dentro de la petición, con riesgo de cache stampede.

El limiter (limiter.py) usa almacenamiento en memoria, así que con 2 workers el límite efectivo se duplica y con más réplicas deja de funcionar. Hay que pasarlo a Redis. Además, --proxy-headers solo confía en 127.0.0.1 por defecto: si hay un balanceador en otro host, todos los clientes pueden compartir IP. Configurar --forwarded-allow-ips.

● Pool de conexiones por defecto. database.py:8-12 no configura nada:

pool_size=5 y max_overflow=10 por proceso. Los endpoints síncronos corren en un threadpool de 40 hilos, y cada uno retiene una conexión durante toda la petición. Bajo carga hay riesgo de QueuePool timeout (a verificar con una prueba de carga). Definir pool_size, max_overflow, pool_timeout y pool_recycle según workers × hilos.

● Sesión y autenticación.

- JWT de 7 días sin refresh, revocación ni logout.
- `get_current_user` no comprueba estado (no hay `is_active` ni baneo).
- Sin `iss` ni `aud`.
- El mismo `SECRET_KEY` firma el JWT y alimenta el HMAC del OTP.
- El registro no verifica el correo, así que se pueden "ocupar" usernames y emails ajenos.
- `register` devuelve un 409 distinto si el usuario existe, lo que contradice la política anti-enumeración de `request-otp`.

● **Manejo global de errores.** El handler de `IntegrityError` (`main.py:40-50`) convierte *cualquier* violación (`NOT NULL`, `FK`, `unique`) en "conflicto con registro existente". Eso ocultó el problema de `phone_number`. Tampoco hay handler para `Exception` o `SQLAlchemyError`, ni `request-id`, ni `logging.basicConfig` en la app, ni logs estructurados. Además `exc.orig` puede contener valores (email, teléfono) y se escribe a logs.

● **Colecciones: API incompleta.**

- `CollectionCreate` no tiene `is_public_trade` ni `art_url`, así que los `getattr(payload, "is_public_trade", True)` de `collection_service.py:48-53` siempre devuelven el default. **No hay forma de hacer privado un binder por API.**
- No existen `PATCH` ni `DELETE` de colección, ni `PATCH` de `UserCard` (cantidad, condición, `is_for_trade`).
- `add_card` activa `is_for_trade` pero nunca lo desactiva.
- `condition` y `language` aceptan cualquier string, porque `set_condition` valida pero no se usa.
- `TradeService` (muro público) no tiene endpoint: código muerto.
- Borrar una `UserCard` referenciada por una propuesta falla por `FK` sin `ondelete` y se convierte en un 409 opaco.

● **Decks.**

- `DeckCard.change_category` (`deck.py:194-198`) omite `companion`, que el schema sí permite. Un `PATCH category=companion` devuelve 400.

- LegalityStrategyFactory aplica reglas de Commander a cualquier formato desconocido (Legacy, Pauper, etc.).
- La "legalidad" no consulta legalities ni las cartas baneadas, que ya están en scryfall_raw_data.
- bulk_add_cards (deck_service.py:89-115) hace una consulta por carta, usa except Exception silencioso y no tiene tope (BulkAddCardsPayload.cards sin max_length), lo que permite un DoS.

● **Schemas duplicados con efecto silencioso.** TradeMarketItemResponse, MatchedCard y TradeMatchUserResponse existen duplicados con campos distintos (schemas/wishlist.py, collection.py y trade.py). El router de matchmaking declara el response_model desde app.schemas (versión de wishlist), pero el servicio construye el de trade.py. FastAPI re-valida contra el primero y **descarta quantity y set_code** (a verificar con un test).

● **Scripts y despliegue.**

- app/scripts/seed_test_user.py queda dentro de la imagen y genera JWT de 30 días: moverlo fuera del paquete de producción.
- ingest_prices.py solo ingesta CardKingdom, pero la API y los regex anuncian tcgplayer, que siempre devuelve null.
- Los JSONDecodeError de la ingesta se ignoran sin log.
- Los precios están solo en USD y preferred_currency se guarda pero no se usa.
- No hay scheduler para las ingestas.

6. Seguridad

Área	Estado	Detalle
SQL injection	●	ORM parametrizado; escape_like en todos los ILIKE; la clave de legalities[...] va como bind
Mass assignment	●	UserProfileUpdate es lista blanca; model_dump(exclude_unset=True)

Área	Estado	Detalle
IDOR / PII	●	Filtros por user_id, 404 opaco, teléfono solo tras ACCEPTED, owner_phone eliminado del muro
OTP	● / ●	Criptografía correcta (secrets, HMAC, compare_digest, TTL 5 min), pero inutilizable (ver 4.2)
JWT	● / ●	algorithms=[...] fijado y require exp,sub, pero sin revocación ni iss/aud
Rate limiting	●	Solo en auth, por IP y en memoria
CORS	●	Orígenes explícitos con credentials. Los defaults son localhost: en prod hay que fijar CORS_ORIGINS (pydantic-settings espera JSON, p. ej. ["https://app.com"])
Secretos	●	El zip trae .env (con credenciales de BD) y no hay .gitignore visible ni .env.example. El .env tampoco incluye SECRET_KEY
Logs	●	request_otp registra el email en INFO; exc.orig puede incluir datos personales
Dependencias	?	No puedo evaluarlas: todas usan >= y no hay lockfile. Ejecutar pip-audit sobre un lock
Cabeceras / HTTPS	●	No hay cabeceras de seguridad ni límite de tamaño de body. Delegable al proxy, pero hay que documentarlo
Exposición	●	/decks/{id}, /decks/{id}/metrics y /decks/users/{id} son públicos y no hay campo de visibilidad. Con esta información no puedo saber si es intencional

7. Base de datos

- Buenas decisiones:
 - Índice trigram en name.
 - UniqueConstraints bien pensados.
 - Numeric para dinero.

- El unique de histórico (card, tienda, tipo, fecha) sirve para ORDER BY fecha DESC.
- **Eager loading con JSONB:** ver sección 5. Impacto en memoria y ancho de banda; solución con deferred() y lazy="select".
- **N+1 en trade:** build_response_dto accede a proposal.receiver y proposal.proposer de forma lazy en propuestas aceptadas, dos consultas extra por propuesta en /proposals/me. get_user_proposals tampoco pagina. Usar selectinload o joinedload.
- **Lost update en cantidades:** existing.quantity += quantity (collection.py:147, deck.py:233) es read-modify-write. Dos peticiones concurrentes pierden incrementos. Usar UPDATE ... SET quantity = quantity + :q o INSERT ... ON CONFLICT DO UPDATE.
- **Cuotas con TOCTOU:** el conteo y luego el insert para el límite de 10 colecciones/mazos no es atómico.
- **Sin CheckConstraints:** quantity > 0, rating entre 0 y 5, cash_amount >= 0. status, side, condition y category son strings libres. Usar Enums con CHECK.
- **Otras:**
 - Los IDs son String en vez de UUID nativo.
 - Faltan ondelete en varias FK.
 - decks.featured_card_id no tiene FK.
 - TradeProposalItem.quantity es Numeric(10,0) en vez de Integer.
 - colors es un CSV en un String, cuyo índice btree no sirve para LIKE '%X%'.
 - Los índices btree sobre rarity, colors, type_line y cmc aportan poco y encarecen la ingesta.
 - ix_cartas_id es redundante con la PK.
 - User.wishlist y wishlist_items duplican la relación.
- **price_service.get_current_prices_summary** hace 5 consultas (1 de existencia y 4 precios); una sola con DISTINCT ON basta. Optimización de bajo impacto.

- **Parser de Scryfall** (scryfall_query_parser.py:16-41): and_(is_numeric, cast(...)) no garantiza orden de evaluación en PostgreSQL y puede dar error 500 con pow>=2 (a verificar). Usar CASE WHEN. Los tokens desconocidos se descartan sin aviso (foo:bar devuelve las primeras 24 cartas), id: se trata como colors, y c>= / c<= ignoran el operador.

8. API

Endpoint	Hallazgo	Propuesta
API_V1_STR = "/api"	No hay versión real	/api/v1
PUT /users/me/profile	Semántica PATCH (exclude_unset)	PATCH
POST /collections/me, GET /decks/users/{id}	Alias duplicados ocultos	Elegir una ruta y mantener la otra con deprecated
DELETE colecciones y mazos	200 con JSON; DELETE /wishlist devuelve 204	Unificar en 204
POST /decks/{id}/cards, PATCH, /metrics	Devuelven dict sin response_model; el POST responde 201 aunque incremente una carta existente	Schemas de respuesta; 200 si fue merge
POST /decks/{id}/fork	new_name como query param sin validar	Body con Field(max_length=100)
Listados /decks/me, /collections/me, /wishlist/me, /trade/proposals/me	Sin paginación	limit/offset y total
/cards/search	Sin offset ni total; orden fijo por nombre	offset + X-Total-Count
/collections/{id}/cards	Sin total	X-Total-Count

Endpoint	Hallazgo	Propuesta
Altas de cartas	Sin idempotencia: un doble submit duplica la cantidad (el 409 no ayuda porque no viola unicidad)	Idempotency-Key o endpoint PUT por clave natural
/users/	Lista todos los usuarios a cualquier autenticado	Aceptable para una comunidad P2P; añadir rate limit

Lo que está bien: verbos accept/reject/complete como sub-recursos de acción, validación con Query(ge/le), pattern en categorías y respuestas 404 coherentes. ●

9. Testing

Qué hay: unas 33 pruebas recolectables en 8 archivos. Los 4 tests de tests/est_domain_collection.py **nunca se ejecutan** por el typo est_. Lo mejor son los 7 tests de test_routers_security.py (PII, enumeración, IDOR). El resto cubre dominio de mazos, inventario, matchmaking e inventario con mocks.

Qué falta:

- Trade (0 tests), flujo AuthService completo, wishlist, usuarios (update de perfil), búsqueda/autocomplete/parser y rate limit (0 tests).
- Todo corre contra **SQLite**, que enmascara JSONB, trigram, with_for_update, comportamiento naive/aware y las migraciones.
- Los tests de dominio dependen de MagicMock que inventan atributos (caso color_identity).
- Sin factories, sin cobertura configurada, sin pytest.ini.

Tests prioritarios:

1. alembic upgrade head + alembic check sobre PostgreSQL.
2. OTP de punta a punta con un OtpSender falso: request → verify OK, 2 intentos fallidos, expiración, código reutilizado.
3. Registro sin teléfono.

4. Trade: no se puede ofrecer carta ajena ni pedir una privada, matriz de transiciones, doble complete idempotente, solo el receptor acepta.
 5. Commander con filas reales de CartaScryfall (identidad de color).
 6. Búsqueda: parser con pow/tou, tokens desconocidos, cadenas de inyección, y contra Postgres.
 7. Colecciones: merge de cantidades concurrente y visibilidad.
 8. Bulk add: tope de tamaño y número de queries (listener de eventos).
 9. Rate limit: 429.
 10. Test del normalizador de ingesta (ScryfallCardNormalizer, función pura y fácil de probar).
-

10. Calidad del código

- **Nombres mezclados** español/inglés: CartaScryfall, HistoricoPrecio, tienda, tipo, puntos frente al resto en inglés. Elegir un idioma, sobre todo en la API pública.
- **Magic strings:** las categorías de mazo aparecen en 5 lugares (regex del schema, change_category, estrategias, CANONICAL_ACTIVE_CATEGORIES, servicio), y también "offered"/"requested", estados de disponibilidad y condiciones. Usar Enum.
- **Defensivo innecesario:** getattr(payload, "x", default) sobre schemas tipados (collection_service, deck_service, wishlist_service, trade_service) y hasattr(UserCard, "created_at"). Además de ruido, enmascara bugs: es lo que oculta que CollectionCreate no tiene is_public_trade.
- **Código muerto:** TradeService, la rama "type" de autocomplete (el regex del router no la deja pasar), el fallback de uid en _resolve_reverse_wants, y DynamicVocabularyCache.invalidate (la ingesta es otro proceso y nunca la invoca; solo expira el TTL de 12 h).
- **Validación inconsistente en wishlist:** priority solo se valida al actualizar, no al crear (wishlist_service.py), y las cantidades acumuladas pueden superar el le=999.

- **Docstrings engañosos:** `get_user_profile` ("sin filtrar PII") y `create_proposal` ("valida que el proponente posea las cartas").
- **Ingesta:** `_extract_colors` usa `color_identity` como fallback de `colors`, lo que mezcla dos conceptos (una tierra con identidad G queda con color G).
- **Dependencias:** `pydantic` y `pydantic[email]` + `email-validator` redundantes; `pytest` y `httpx` están en el `requirements` de runtime.
- **Docker:** el `HEALTHCHECK` usa `/health/ready`; si la BD cae, el orquestador reinicia todos los contenedores en cascada. Usar `/health/live` ahí y reservar `ready` para el balanceador.

11. Deuda técnica

Prioridad	Archivo / método	Problema	Principio	Riesgo	Solución
●	<code>alembic/versions/*</code> , <code>models/__init__.py</code>	Migraciones desfasadas, trade sin tablas	Mantenibilidad	App rota en BD migrada	Migración correctiva + alembic check en CI
●	<code>auth_service.requestotp</code>	OTP nunca se envía	Compleitud	Nadie puede iniciar sesión	<code>OtpSender</code> + proveedor
●	<code>user.py:56-79</code>	Naive vs aware	Corrección	500 en <code>verify-otp</code>	<code>_utc()</code> o <code>timezone=True</code>
●	<code>security.py:59</code> + <code>user.py:68</code>	Doble conteo y límites 3/5	SRP / DRY	Bloqueo injusto, griefing	<code>User.verify_otp</code> único
●	<code>trade_proposal_service.py</code>	Sin propiedad, sin inventario,	Encapsulación	Integridad y reputación manipulables	Validación + transicione

Prioridad	Archivo / método	Problema	Principio	Riesgo	Solución
		sin estados, sin locks			s + FOR UPDATE
●	deck.py:67-118, card.py	color_identity y inexistente	Corrección	Validación Commander nula	Columna + tests reales
●	deck.py, collection.py	Eager loading con JSONB	Rendimiento	Memoria y latencia	deferred + lazy="select"
●	routers/cards.py, limiter.py	Endpoints costosos sin límite; limiter en memoria	Seguridad	DoS	Redis + límites + caché
●	main.py:40-50	IntegrityError genérico y sin handler 500	Manejo de errores	Bugs ocultos	Excepciones de dominio + logging
●	database.py	Pool por defecto	Escalabilidad	Timeouts bajo carga	Dimensionar pool
●	collection_service.py, routers/collections.py	API incompleta (privacidad, PATCH/DELETE)	YAGNI inverso	Funciones inalcanzables	Completar CRUD
●	deck_service.bulk_add_cards	N queries, sin tope	Rendimiento	DoS	IN + max_length
●	schemas/wishlist.py vs trade.py	Duplicados	DRY	Pérdida silenciosa de campos	Un solo schema

Prioridad	Archivo / método	Problema	Principio	Riesgo	Solución
	tests/est_domain_collection.py	Typo en el nombre	Calidad	4 tests muertos	Renombrar
	Varios	Nombres ES/EN, magic strings, getattr defensivo	Clean Code	Mantenibilidad	Enums y limpieza

12. Plan de mejora

Fase 1: Correcciones críticas

- Migración correctiva y test de migraciones.
- User.verify_otp único y OtpSender.
- Validación de propiedad en trade y tabla de transiciones.

Fase 2: Arquitectura

- Excepciones de dominio con handler HTTP.
- Unit of Work en get_db.
- deferred y lazy="select".
- Extraer la heurística de CardRepository.

Fase 3: Calidad

- Enums, schemas deduplicados y limpieza de getattr defensivo.
- Completar CRUD de colecciones.
- Un idioma para la API.

Fase 4: Testing

- PostgreSQL en CI.
- Los 10 tests prioritarios.

- Factories reales y cobertura configurada.

Fase 5: Seguridad

- Rate limit en Redis.
- Revocación de JWT o tokens cortos con refresh.
- Verificación de correo en registro.
- pip-audit sobre un lockfile.
- Cabeceras de seguridad y logs sin PII.

Fase 6: Optimización

- Índices para oracle_text y colors, o columnas normalizadas.
- Caché para /similar y vocabularios.
- UPDATE ... SET quantity = quantity + :q.
- Paginación y totales.

Fase 7: Producción

- README, .env.example, .gitignore.
- Dependencias fijadas, CI, imagen escaneada.
- Migraciones como paso de despliegue.
- HEALTHCHECK en liveness, pool y workers dimensionados.
- Logs estructurados y monitoreo de errores.
- seed_test_user.py fuera de la imagen.

Criterio final

¿Está técnicamente terminado?

Requiere correcciones antes de producción.

Los bloqueos son errores y huecos funcionales localizados (migraciones, OTP, trade), no fallos de diseño. La arquitectura en capas es adecuada y no requiere cambios estructurales. Es el extremo alto de esa categoría, porque los tres bloqueos impiden operar el sistema.

Top 10 acciones recomendadas

1. **Alinear migraciones con modelos** y añadir alembic check en CI. Desbloquea mazos, trade y registro sin teléfono.
 2. **Implementar el envío de OTP** con una interfaz OtpSender. Sin esto no hay login en producción.
 3. **Corregir User.verify_otp**: fechas aware, un solo contador y un solo límite.
 4. **Validar propiedad y disponibilidad en create_proposal**. Cierra la manipulación de intercambios.
 5. **Máquina de estados del trade** con bloqueo de fila, confirmación bilateral e impacto real en inventario.
 6. **color_identity real** y tests con modelos reales. Hace efectiva la validación de Commander.
 7. **deferred para scryfall_raw_data y quitar los lazy="joined" globales**. Mayor ganancia de rendimiento por menor esfuerzo.
 8. **Rate limiting en Redis** más límites en las rutas de búsqueda, similares y autocompletado.
 9. **Excepciones de dominio, handler genérico y logging** en lugar del 409 universal y los HTTPException en servicios.
 10. **Tests contra PostgreSQL en CI**, más README, lockfile y .env.example.
-

Información que me faltó

- **Cómo se crea y migra la BD en la práctica** (no hay README ni docker-compose).
- **Intención de privacidad de mazos** y del endpoint de listado de usuarios.
- **Despliegue real**: proxy y balanceador, número de réplicas.
- **Contrato con el frontend**.
- **Lockfile de dependencias**.
- **Resultados reales de la suite**.